

Linear Regression & Optimization

A comprehensive VS Code implementation guide: mathematics, formulas, visualizations, files, experiments, commands, and optimizer derivations

Project focus	Tools	Scope
Linear Regression + Optimization	Python, NumPy, Matplotlib	From model definition through Adam
Learning style	Derivation + implementation + visualization	No Scikit-learn for the core algorithms
Environment	VS Code / Windows / PowerShell	Runnable file-by-file experiments

End point: Adam. Optimization section intentionally stops here.

Contents

- 1. Project overview and learning path
- 2. Environment and setup
- 3. Dataset and first visualization
- 4. Linear Regression model
- 5. Residuals and MSE
- 6. Gradient derivation
- 7. One Gradient Descent step
- 8. Full Gradient Descent training loop
- 9. Loss vs iteration and animation
- 10. Loss surface and parameter geometry
- 11. Gradient vector and gradient checking
- 12. Learning rate and convergence
- 13. Hessian, eigenvalues, and stability
- 14. Feature scaling and conditioning
- 15. Parameter trajectories
- 16. Momentum and beta
- 17. Nesterov Accelerated Gradient
- 18. AdaGrad
- 19. RMSProp
- 20. Adam
- 21. Optimizer comparison
- 22. File-by-file guide
- 23. All terminal commands
- 24. Reproducibility and troubleshooting
- 25. Formula sheet
- 26. What was completed / stopping point

1. Project Overview and Learning Path

This project treats Linear Regression as a vehicle for understanding optimization from first principles. The workflow is intentionally incremental: first define a model, then define a loss, differentiate it, implement Gradient Descent, visualize what it is doing, analyze stability and conditioning, and finally implement momentum-based and adaptive optimizers.

Data -> Model -> Prediction -> Loss -> Gradient -> Update -> Repeat -> Convergence

Stage	Core idea	Main artifact
Linear Regression	$\hat{y} = b_0 + b_1 \cdot x$	linear_regression.py
Loss	MSE measures squared prediction error	linear_regression.py
Gradient	Partial derivatives of MSE	gradient_vector.py
Gradient Descent	Move opposite the gradient	linear_regression.py
Geometry	Loss surfaces and parameter path	loss_surface.py / loss_surface_3d.py
Verification	Finite-difference gradient check	gradient_check.py
Stability	Learning rate, Hessian, eigenvalues	hessian_analysis.py / stability_boundary.py
Conditioning	Feature scaling changes curvature	feature_scaling.py
Acceleration	Momentum and Nesterov	momentum_gradient_descent.py / nesterov_momentum.py
Adaptive methods	AdaGrad, RMSProp, Adam	adagrad.py / rmsprop.py / adam.py

2. Environment and Setup

The implementation was built interactively in VS Code on Windows using PowerShell. Python was verified first, then NumPy and Matplotlib were installed. The core algorithms are intentionally implemented without Scikit-learn.

```
python --version
python -m pip install numpy
python -m pip install matplotlib
```

For a reproducible dependency setup, the two packages can also be installed together:

```
python -m pip install numpy matplotlib
```

Directory location used in the project

```
C:\Users\Lenovo\machine-learning-algorithms\01_linear_regression
```

A recurring VS Code terminal issue was simply being one directory above the Python file. The reliable fix was to enter the experiment folder before executing the script:

```
cd .\01_linear_regression
python linear_regression.py
```

3. Dataset and First Visualization

The training data used throughout the project is a small synthetic relationship between hours studied and exam score.

X: hours studied	y: exam score
1	35
2	42
3	51
4	58
5	66
6	73

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([1, 2, 3, 4, 5, 6], dtype=float)
y = np.array([35, 42, 51, 58, 66, 73], dtype=float)

plt.scatter(X, y)
plt.xlabel("Hours Studied")
plt.ylabel("Exam Score")
plt.title("Training Data")
plt.show()
```

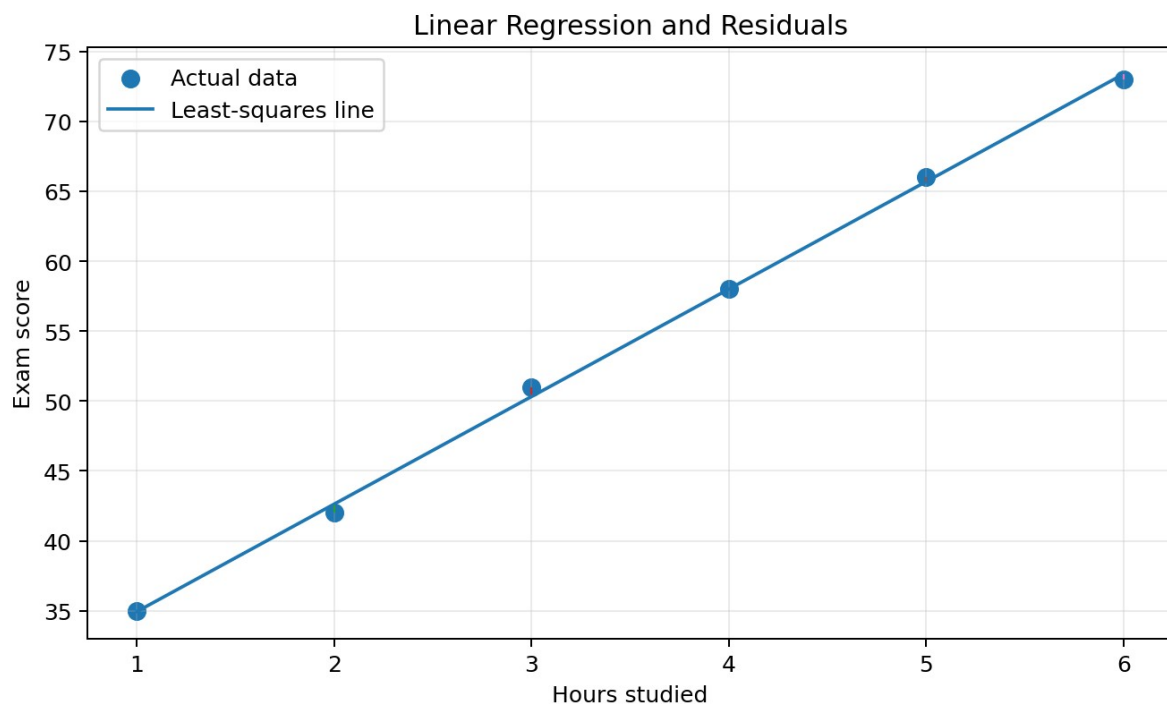


Figure 1. Actual data, least-squares line, and vertical residuals.

4. Linear Regression Model

The first model is a one-feature linear predictor. The intercept b_0 shifts the line vertically; the slope b_1 determines how much the prediction changes for one unit of x .

$$y_{\text{hat}_i} = b_0 + b_1 * x_i$$

$$\theta = [b_0, b_1]^T$$

At the beginning, b_0 and b_1 can be initialized arbitrarily. In the project they were initialized at zero for Gradient Descent experiments.

```
b0 = 0.0
b1 = 0.0
y_pred = b0 + b1 * X
```

5. Residuals and Mean Squared Error

A residual is the signed vertical prediction error. Squaring the residual prevents positive and negative errors from cancelling and penalizes larger errors more strongly.

$$e_i = y_i - y_{\text{hat}_i}$$

$$\text{MSE} = J(b_0, b_1) = (1/n) * \sum_i [(y_i - y_{\text{hat}_i})^2]$$

$$J(b_0, b_1) = (1/n) * \sum_i [y_i - (b_0 + b_1 * x_i)]^2$$

For the intentionally poor line $b_0 = 0$, $b_1 = 5$, the project produced:

Quantity	Value
Predictions	[5, 10, 15, 20, 25, 30]
Residuals ($y - y_{\text{hat}}$)	[30, 32, 36, 38, 41, 43]
MSE	1365.6666666667

The MSE is the scalar quantity the optimizer tries to minimize.

6. Gradient Derivation

The gradient is the vector of partial derivatives of the loss with respect to the parameters.

$$\text{grad } J = [dJ/db_0, dJ/db_1]^T$$

6.1 Derivative with respect to b_0

$$J = (1/n) * \sum_i (y_i - b_0 - b_1 * x_i)^2$$

$$dJ/db_0 = -(2/n) * \sum_i [y_i - (b_0 + b_1 * x_i)]$$

$$\text{Equivalent implementation form: } dJ/db_0 = (2/n) * \sum_i (y_{\text{hat}_i} - y_i)$$

6.2 Derivative with respect to b1

$$dJ/db1 = -(2/n) * \sum_i x_i * [y_i - (b0 + b1*x_i)]$$

Equivalent implementation form: $dJ/db1 = (2/n) * \sum_i x_i * (y_{\hat{i}} - y_i)$

6.3 Numerical example from the project

At $b0 = 0$ and $b1 = 5$, the residuals $y - y_{\hat{}}$ are [30, 32, 36, 38, 41, 43]. The gradients are therefore:

Gradient component	Computed value
dJ/db0	-73.3333333333
dJ/db1	-272.3333333333

Both components are negative, so subtracting the gradient increases both parameters on the next descent step.

7. One Gradient Descent Step

Gradient Descent moves in the direction opposite the gradient. The scalar alpha controls the step length.

$$\theta_{t+1} = \theta_t - \alpha * \text{grad } J(\theta_t)$$

$$b0_{\text{new}} = b0 - \alpha * dJ/db0$$

$$b1_{\text{new}} = b1 - \alpha * dJ/db1$$

With $\alpha = 0.01$, $b0 = 0$, $b1 = 5$, the first update is approximately:

Parameter	Before	Gradient	After
b0	0	-73.3333	0.7333
b1	5	-272.3333	7.7233

```
db0 = (-2 / n) * np.sum(y - y_pred)
db1 = (-2 / n) * np.sum(X * (y - y_pred))

b0 = b0 - learning_rate * db0
b1 = b1 - learning_rate * db1
```

8. Full Gradient Descent Training Loop

The one-step update becomes a loop. Each iteration predicts, evaluates MSE, computes the gradient, and updates the parameters.

```
learning_rate = 0.01
epochs = 1000
loss_history = []

for epoch in range(epochs):
    y_pred = b0 + b1 * X
```

```

mse = np.mean((y - y_pred) ** 2)
loss_history.append(mse)

db0 = (-2 / len(X)) * np.sum(y - y_pred)
db1 = (-2 / len(X)) * np.sum(X * (y - y_pred))

b0 -= learning_rate * db0
b1 -= learning_rate * db1

```

Using the same dataset, the analytical least-squares solution is approximately $b_0 = 27.2667$ and $b_1 = 7.6857$ with MSE about 0.18413. Gradient Descent approaches the same optimum asymptotically when configured appropriately.

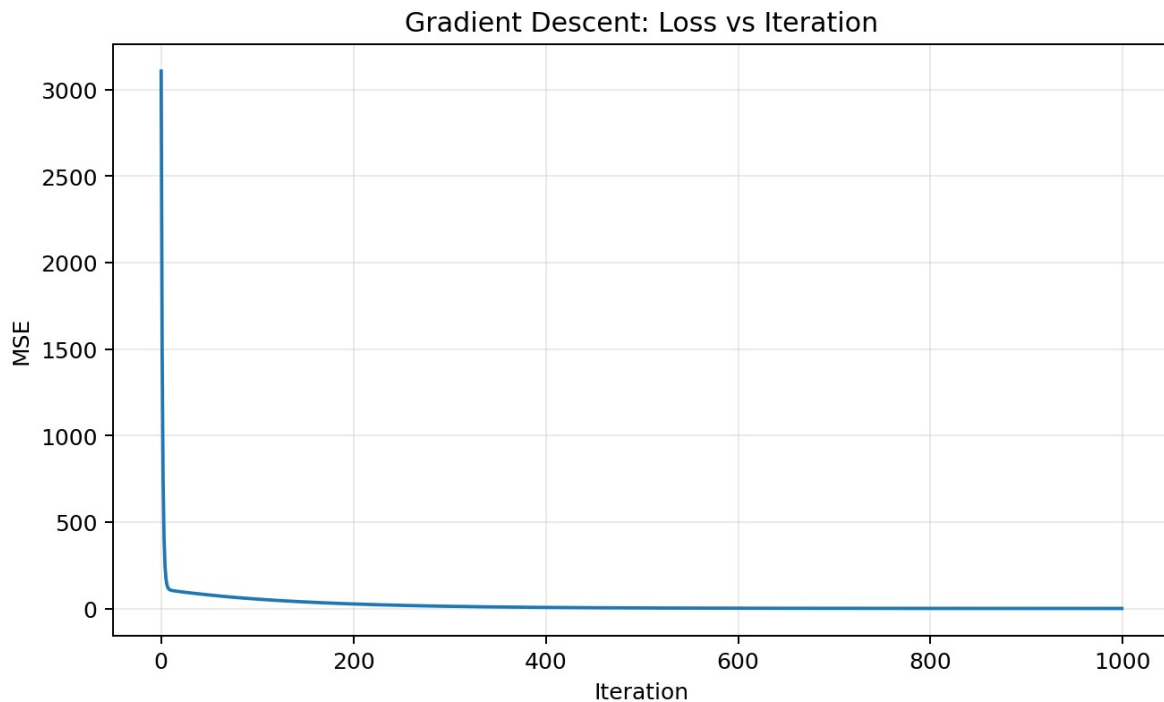


Figure 2. MSE decreases rapidly and then flattens as Gradient Descent approaches the minimum.

9. Loss vs Iteration and Gradient Descent Animation

The project also stores b_0 , b_1 , and loss at every iteration so that the training process can be animated. The animation shows the regression line moving from an initially poor line toward the learned line while the displayed MSE falls.

```

b0_history = []
b1_history = []
loss_history = []

# each iteration:
b0_history.append(b0)
b1_history.append(b1)
loss_history.append(mse)

```

The animation uses Matplotlib FuncAnimation and updates the line, iteration number, b_0 , b_1 , and MSE. Close each Matplotlib window to allow a script containing multiple `plt.show()` calls to continue to the next figure.

10. Loss Surface and Parameter Geometry

Because the model has two parameters, its loss is a surface over the parameter plane (b_0 , b_1). A contour plot shows equal-loss curves; a 3D plot shows the height of the loss surface.

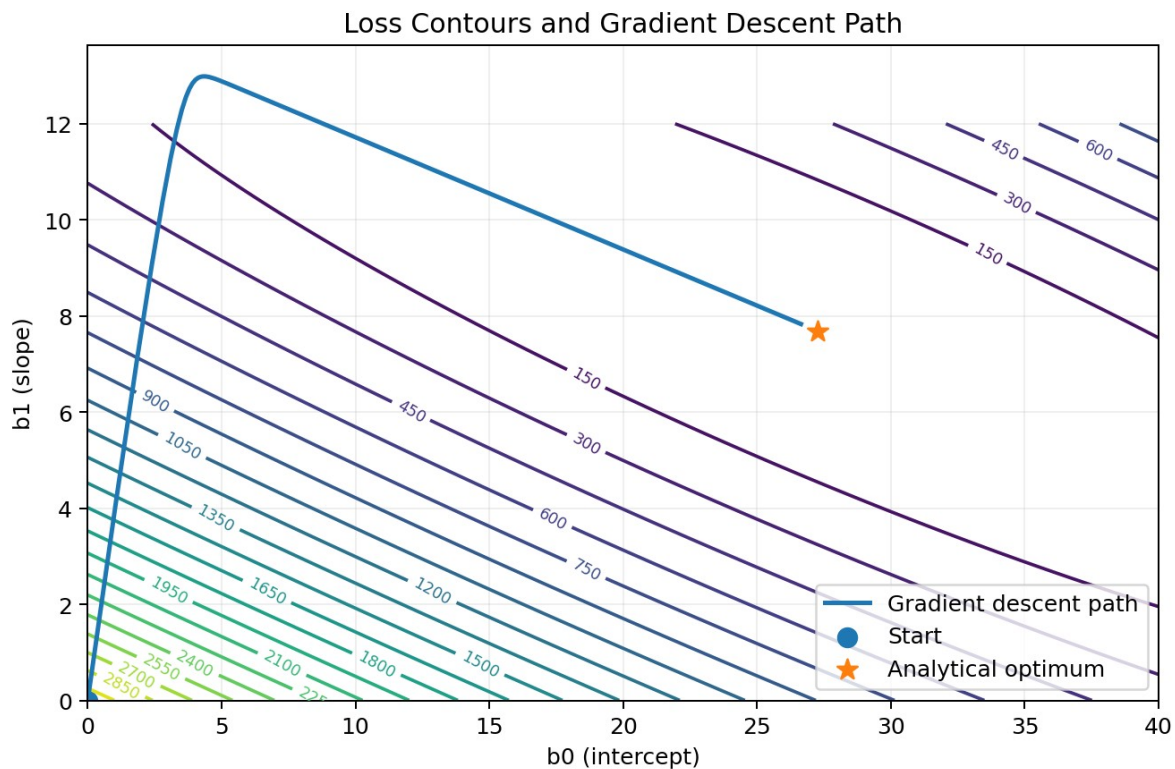


Figure 3. Contours of $J(b_0, b_1)$ and the Gradient Descent path.

3D Loss Surface

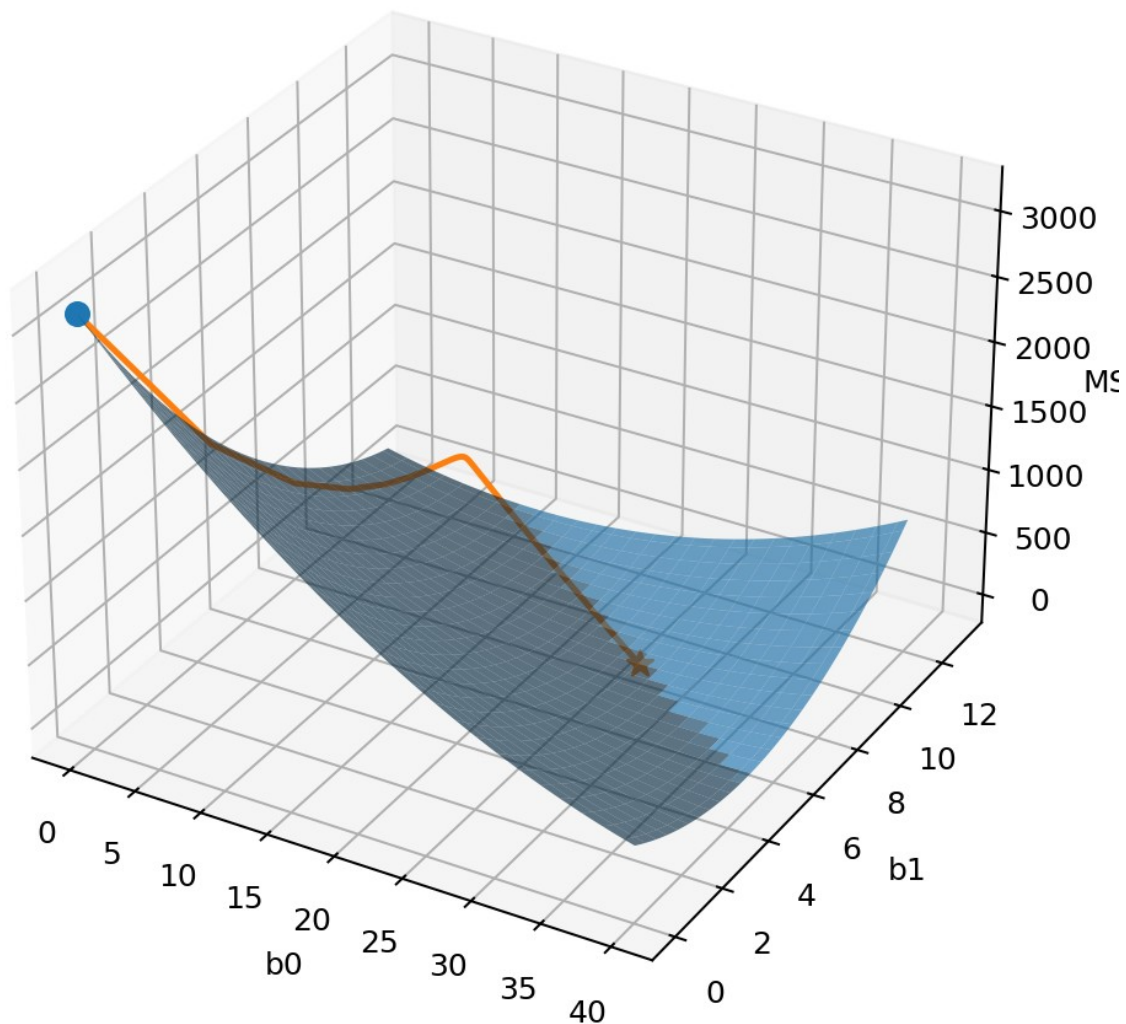


Figure 4. 3D bowl-shaped MSE surface with the optimization path.

The starting point is (0, 0). The path travels toward the bottom of the bowl. The optimum for this least-squares problem is a global minimum under the usual full-rank condition on the design matrix.

11. Gradient Vector and Gradient Checking

At a point in parameter space, the gradient is an arrow in the direction of steepest local increase of the loss. Therefore $-\text{grad } J$ is the local steepest-descent direction.

$\text{grad } J$ = steepest local increase direction

$-\text{grad } J$ = steepest local decrease direction

The project verified this numerically by moving a small amount in the $+\text{grad}$ direction and the $-\text{grad}$ direction. The $+\text{grad}$ move increased MSE; the $-\text{grad}$ move decreased MSE.

11.1 Finite-difference gradient check

$$dJ/d\theta \approx [J(\theta + \epsilon) - J(\theta - \epsilon)] / (2 \cdot \epsilon)$$

Central finite differences provide an independent numerical derivative and are a practical way to debug analytical gradient implementations.

12. Learning Rate and Convergence

The learning rate α controls how far the optimizer moves along the negative-gradient direction. The same model can behave very differently for different α values.

Learning rate	Observed behavior in this project
0.0001	Very slow progress
0.001	Stable, slower convergence
0.005	Fast convergence
0.01	Fast convergence on this dataset
0.05	Stable in the tested regime
0.1+	Can become unstable for this formulation

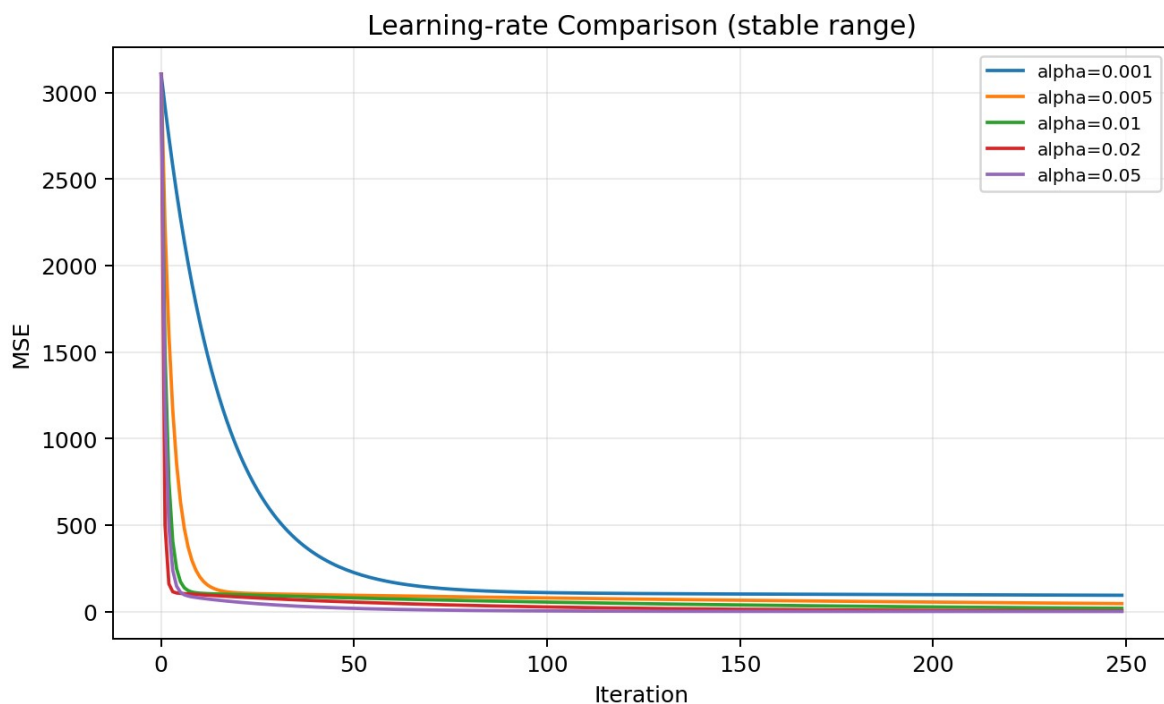


Figure 5. Stable learning-rate comparison.

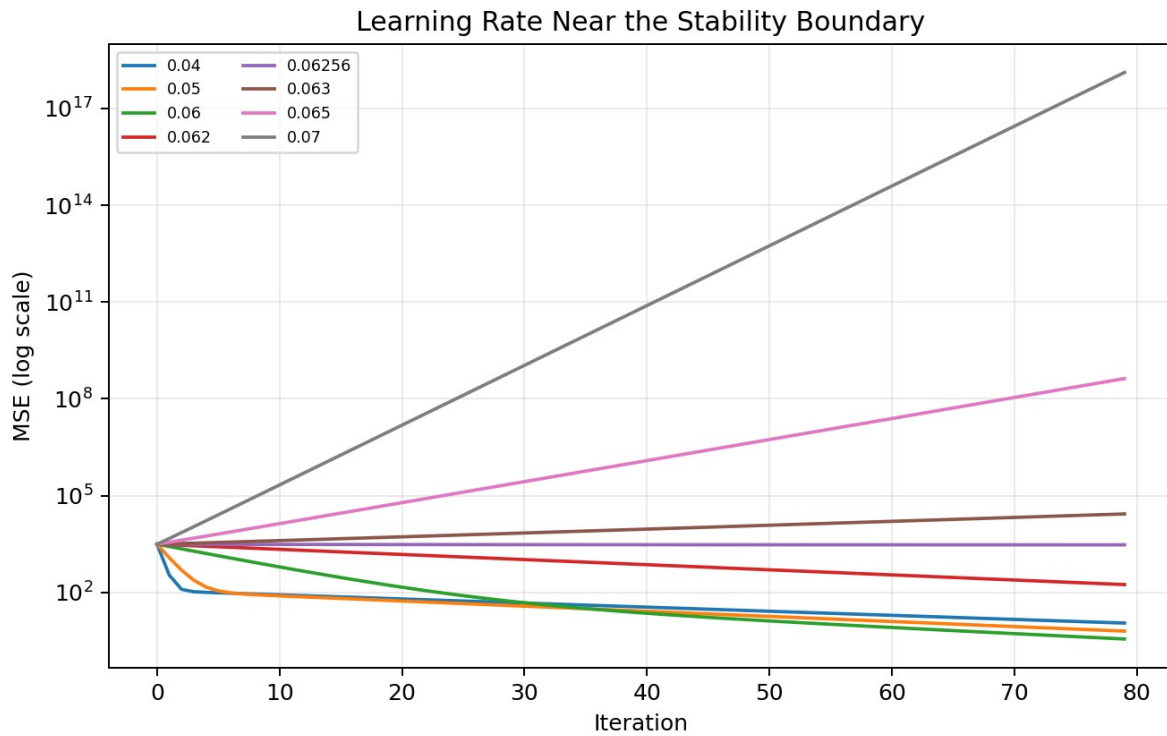


Figure 6. Behavior near the theoretical stability boundary.

A large learning rate can overshoot the minimum. Because the gradient itself becomes large when the model is far from the optimum, an excessively large step can make the next error even larger, creating a positive feedback loop and eventually numerical explosion.

13. Hessian, Eigenvalues, and Stability

For this quadratic MSE objective, the Hessian captures the curvature of the loss surface.

$$H = (2/n) * [[n, \text{sum}(x)], [\text{sum}(x), \text{sum}(x^2)]]$$

For $X = [1, 2, 3, 4, 5, 6]$, the project computed:

Quantity	Value
Hessian H	[[2, 7], [7, 30.33333333]]
Eigenvalues	0.36494384 and 31.96838950
lambda_max	31.96838950
2/lambda_max	0.06256180

For a quadratic objective: $0 < \alpha < 2/\lambda_{\max}(H)$

The experiments matched this prediction: $\alpha = 0.05$ remained stable, while $\alpha = 0.1$ and $\alpha = 0.2$ became unstable for the original feature representation.

14. Feature Scaling and Conditioning

Standardization maps the feature to zero mean and unit standard deviation:

$$x_{\text{scaled}} = (x - \mu) / \sigma$$

For the dataset, $\mu = 3.5$ and σ approximately 1.707825. The scaled values are approximately [-1.46385, -0.87831, -0.29277, 0.29277, 0.87831, 1.46385].

The scaled Hessian is essentially $\text{diag}(2,2)$, with eigenvalues 2 and 2. Hence its condition number is 1. In the unscaled coordinate system, the condition number is approximately 87.6.

Metric	Original feature	Standardized feature
Hessian	[[2,7],[7,30.3333]]	approximately [[2,0],[0,2]]
Eigenvalues	0.36494, 31.96839	2, 2
Condition number	approximately 87.6	1
Theoretical alpha upper bound	approximately 0.06256	approximately 1

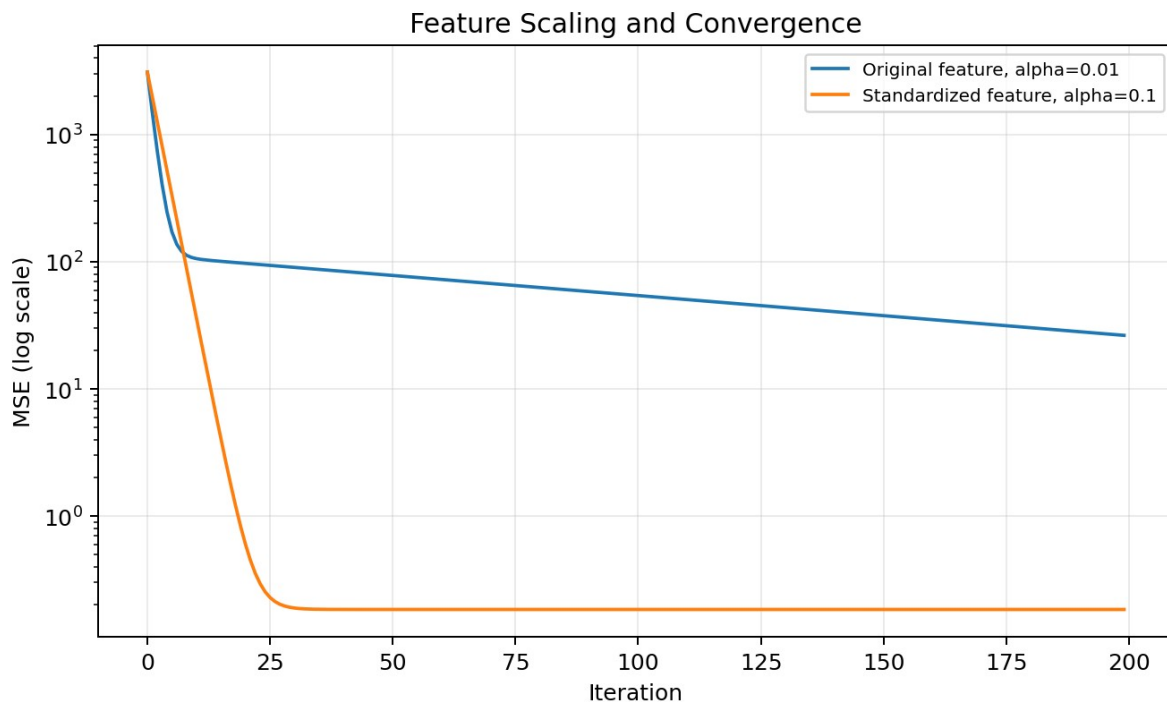


Figure 7. Standardization improves the numerical conditioning of the optimization problem and permits a larger stable step in this example.

Scaling changes the parameterization, not the underlying input-output relationship. The intercept and slope in standardized coordinates differ from those in the original coordinates, but the represented regression line can be transformed back to the original units.

$$\text{If } x_{\text{scaled}} = (x - \mu) / \sigma, \text{ then } b1_{\text{original}} = b1_{\text{scaled}} / \sigma$$

$$b0_{\text{original}} = b0_{\text{scaled}} - (\mu / \sigma) * b1_{\text{scaled}}$$

15. Parameter Trajectories

Tracking (b_0, b_1) at every iteration shows how the optimizer moves through parameter space. Poorly conditioned surfaces tend to produce inefficient, curved or zig-zagging trajectories, while a better-conditioned problem is more direct.

A parameter trajectory is stored with:

```
b0_history.append(b0)
b1_history.append(b1)
```

16. Momentum and the beta Hyperparameter

Momentum augments Gradient Descent with a velocity that retains information from previous updates.

$$v_{(t+1)} = \beta \cdot v_t - \alpha \cdot \text{grad } J(\theta_t)$$

$$\theta_{(t+1)} = \theta_t + v_{(t+1)}$$

When $\beta = 0$, the method reduces to ordinary Gradient Descent. Increasing β increases inertia. The project compared $\beta = 0, 0.5, 0.9$, and 0.99 .

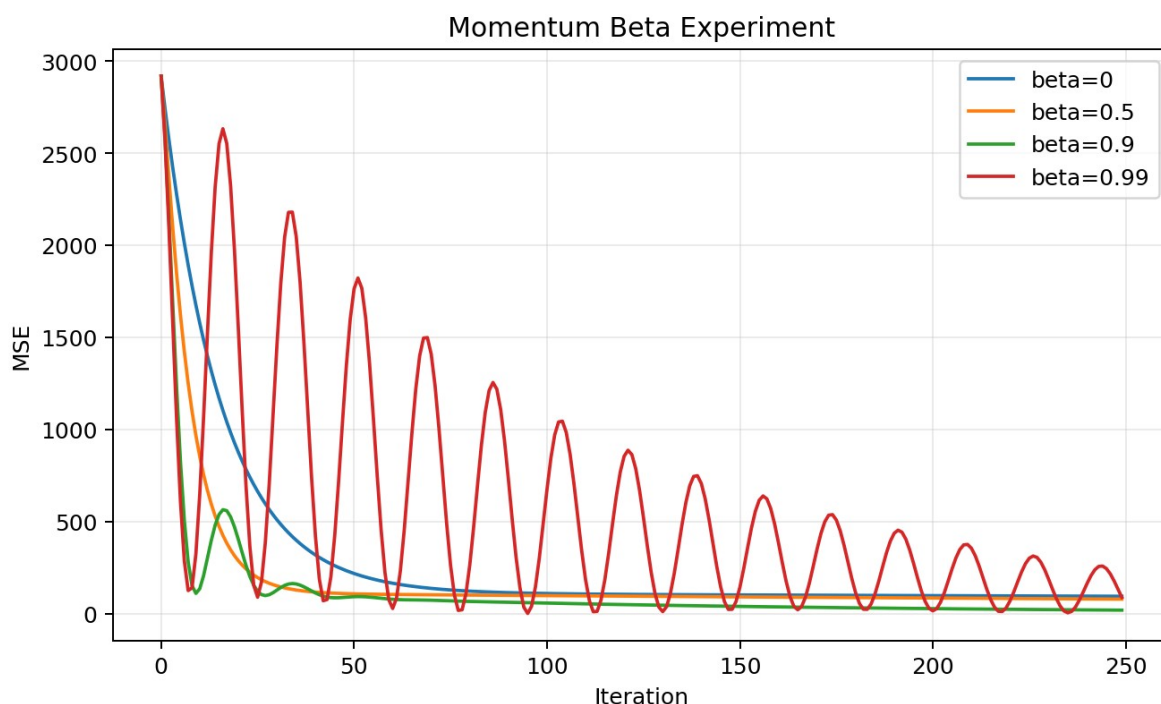


Figure 8. Effect of the momentum coefficient β on loss in this experiment.

The experiment shows that momentum can accelerate motion but can also create overshoot and oscillation when inertia is strong relative to the local curvature and step size.

17. Nesterov Accelerated Gradient

Nesterov modifies momentum by evaluating the gradient at a look-ahead point. This anticipates the direction produced by the current velocity before computing the correction.

$$\text{theta_lookahead} = \text{theta_t} + \text{beta} * \text{v_t}$$

$$\text{g_t} = \text{grad } J(\text{theta_lookahead})$$

$$\text{v_t}(t+1) = \text{beta} * \text{v_t} - \text{alpha} * \text{g_t}$$

$$\text{theta_t}(t+1) = \text{theta_t} + \text{v_t}(t+1)$$

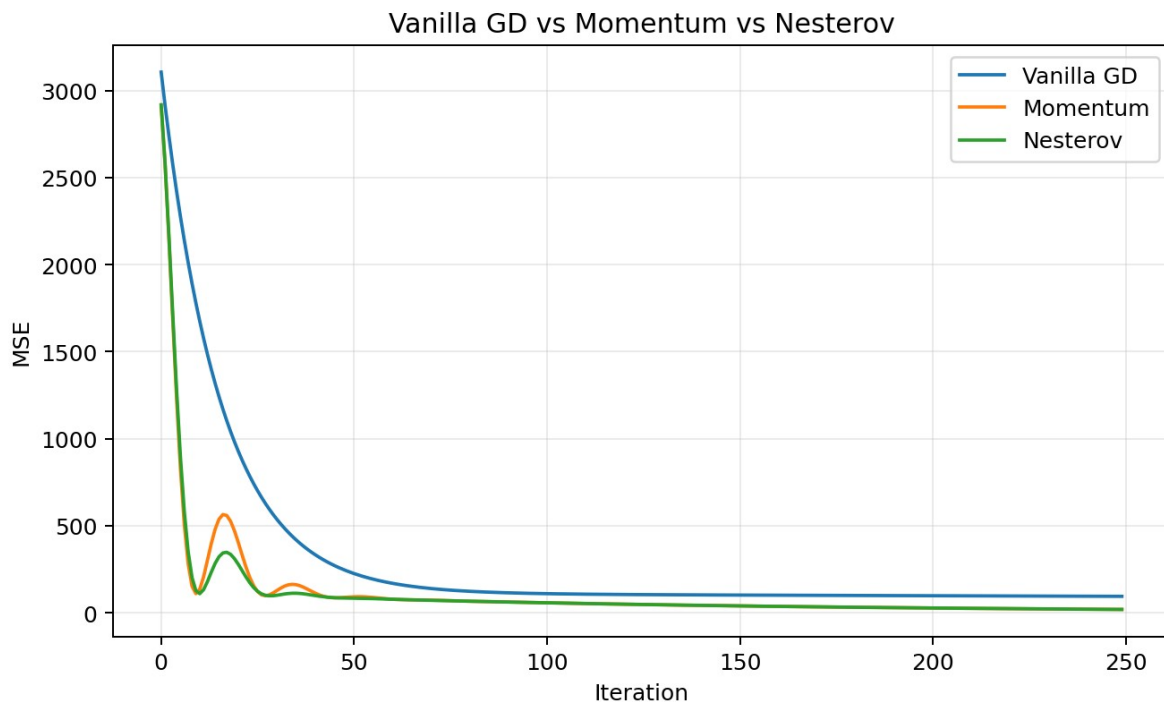


Figure 9. Vanilla Gradient Descent, Momentum, and Nesterov comparison.

The key conceptual difference is where the gradient is evaluated: current position for ordinary momentum versus look-ahead position for Nesterov.

18. AdaGrad

AdaGrad adapts the learning rate separately for each parameter using an accumulator of squared gradients.

$$\text{G_t} = \text{G_t}(t-1) + \text{g_t}^2$$

$$\text{theta_t}(t+1) = \text{theta_t} - \text{alpha} * \text{g_t} / (\text{sqrt}(\text{G_t}) + \text{epsilon})$$

This makes the effective learning rate smaller for parameters whose gradients have accumulated large squared magnitude. The drawback is that G_t is monotonic non-decreasing, so the learning rate can eventually become excessively small.

```
# AdaGrad core
G_b0 += db0**2
G_b1 += db1**2
b0 -= alpha * db0 / (np.sqrt(G_b0) + epsilon)
b1 -= alpha * db1 / (np.sqrt(G_b1) + epsilon)
```

19. RMSProp

RMSProp replaces AdaGrad's permanent accumulation with an exponential moving average of squared gradients.

$$S_t = \beta S_{(t-1)} + (1-\beta)g_t^2$$

$$\theta_{(t+1)} = \theta_t - \alpha * g_t / (\sqrt{S_t} + \epsilon)$$

Recent squared gradients receive more weight, while older gradients gradually lose influence. This prevents the denominator from growing without bound in the same way as AdaGrad.

```
# RMSProp core
S_b0 = beta*S_b0 + (1-beta)*db0**2
S_b1 = beta*S_b1 + (1-beta)*db1**2
b0 -= alpha * db0 / (np.sqrt(S_b0) + epsilon)
b1 -= alpha * db1 / (np.sqrt(S_b1) + epsilon)
```

20. Adam

Adam combines a first-moment estimate of the gradient, a second-moment estimate of the squared gradient, and bias correction.

$$m_t = \beta_1 m_{(t-1)} + (1-\beta_1)g_t$$

$$v_t = \beta_2 v_{(t-1)} + (1-\beta_2)g_t^2$$

$$m_{\text{hat}}_t = m_t / (1-\beta_1^t)$$

$$v_{\text{hat}}_t = v_t / (1-\beta_2^t)$$

$$\theta_{(t+1)} = \theta_t - \alpha * m_{\text{hat}}_t / (\sqrt{v_{\text{hat}}_t} + \epsilon)$$

Typical values used in the implementation are $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 1e-8$. The first moment supplies direction/history similar to momentum; the second moment supplies parameter-wise scaling similar to RMSProp.

```
# Adam core
m_b0 = beta1*m_b0 + (1-beta1)*db0
m_b1 = beta1*m_b1 + (1-beta1)*db1
```

```

v_b0 = beta2*v_b0 + (1-beta2)*db0**2
v_b1 = beta2*v_b1 + (1-beta2)*db1**2

m_hat_b0 = m_b0 / (1-beta1**t)
m_hat_b1 = m_b1 / (1-beta1**t)
v_hat_b0 = v_b0 / (1-beta2**t)
v_hat_b1 = v_b1 / (1-beta2**t)

b0 -= alpha*m_hat_b0/(np.sqrt(v_hat_b0)+epsilon)
b1 -= alpha*m_hat_b1/(np.sqrt(v_hat_b1)+epsilon)

```

21. Optimization Algorithm Comparison

Algorithm	State maintained	Main idea	Core update
Gradient Descent	None	Use current gradient	$\theta \leftarrow \theta - \alpha * g$
Momentum	Velocity v	Accumulate direction	$v \leftarrow \beta * v - \alpha * g$; $\theta \leftarrow \theta + v$
Nesterov	Velocity v	Look ahead before gradient	$g \leftarrow \text{grad}(\theta + \beta * v)$
AdaGrad	Squared-gradient sum G	Adaptive learning rate from cumulative history	$\theta \leftarrow \theta - \alpha * g / (\sqrt{G} + \epsilon)$
RMSProp	Squared-gradient EMA S	Adaptive learning rate from recent history	$\theta \leftarrow \theta - \alpha * g / (\sqrt{S} + \epsilon)$
Adam	First moment m + second moment v	Momentum + adaptive scaling + bias correction	$\theta \leftarrow \theta - \alpha * \hat{m} / (\sqrt{\hat{v}} + \epsilon)$

Conceptual progression

Gradient Descent -> add velocity -> Momentum

Momentum -> add look-ahead gradient -> Nesterov

Gradient history -> squared-gradient accumulation -> AdaGrad

AdaGrad -> replace permanent sum with EMA -> RMSProp

Momentum + RMSProp-style scaling + bias correction -> Adam

22. File-by-File Guide

File	Purpose	What it demonstrates
linear_regression.py	Main end-to-end baseline	Data, prediction, MSE, Gradient Descent, loss history, plots, animation version
loss_surface.py	2D contours	MSE geometry and Gradient Descent path
loss_surface_3d.py	3D surface	Bowl-shaped objective and descent trajectory
gradient_vector.py	Gradient arrows	$\text{grad } J$ and $-\text{grad } J$ directions
gradient_check.py	Derivative verification	Analytical versus finite-difference reasoning
learning_rate_experiment.py	Hyperparameter study	Slow/stable/unstable learning-rate regimes
hessian_analysis.py	Curvature analysis	Hessian, eigenvalues, theoretical

File	Purpose	What it demonstrates
		stability limit
stability_boundary.py	Boundary experiment	Behavior near $2/\lambda_{\max}$
feature_scaling.py	Scaling study	Standardization, Hessian change, condition number
parameter_trajectory.py	Path comparison	Parameter-space trajectories
momentum_gradient_descent.py	Momentum implementation	Vanilla GD versus velocity-based update
momentum_beta_experiment.py	Beta study	Effect of momentum coefficient
nesterov_momentum.py	NAG implementation	Look-ahead gradient
adagrad.py	Adaptive optimizer	Cumulative squared-gradient accumulator
rmsprop.py	Adaptive optimizer	EMA of squared gradients
adam.py	Adaptive optimizer	First/second moments and bias correction

23. All Terminal Commands

23.1 Enter the project directory

```
cd C:\Users\Lenovo\machine-learning-algorithms\01_linear_regression
```

23.2 Install dependencies

```
python -m pip install numpy matplotlib
```

23.3 Verify NumPy / Matplotlib

```
python -c "import numpy; print(numpy.__version__)"
python -c "import matplotlib; print(matplotlib.__version__)"
```

23.4 Run every project file

```
python linear_regression.py
python loss_surface.py
python loss_surface_3d.py
python gradient_vector.py
python gradient_check.py
python learning_rate_experiment.py
python hessian_analysis.py
python stability_boundary.py
python feature_scaling.py
python parameter_trajectory.py
python momentum_gradient_descent.py
python momentum_beta_experiment.py
python nesterov_momentum.py
python adagrad.py
python rmsprop.py
python adam.py
```

23.5 Useful VS Code workflow

- Open the 01_linear_regression folder, not only the parent repository.
- Use Ctrl+S before running a script.
- Run the script from the integrated terminal so the working directory is visible.

- When a script displays multiple Matplotlib windows sequentially, close the current figure to let execution continue.

24. Reproducibility and Troubleshooting

Symptom	Likely cause	Fix
No module named numpy	NumPy not installed in the active interpreter	<code>python -m pip install numpy</code>
No module named matplotlib	Matplotlib not installed	<code>python -m pip install matplotlib</code>
File not found	Terminal is in parent directory	<code>cd .\01_linear_regression</code>
Second plot never appears	First <code>plt.show()</code> is blocking	Close the current Matplotlib window
Loss explodes	Learning rate too large	Reduce alpha; inspect Hessian stability
Training is extremely slow	Learning rate too small	Increase alpha or scale features
Model values differ after scaling	Parameters are in different coordinate systems	Transform standardized parameters back to original units

Numerical stability is part of the learning objective. Do not treat divergence as merely a software bug: a diverging loss curve can be a mathematically meaningful demonstration of poor hyperparameter choice.

25. Formula Sheet

Concept	Formula
Model	$\hat{y} = b_0 + b_1 x$
Residual	$e_i = y_i - \hat{y}_i$
MSE	$J = (1/n) * \sum_i (y_i - \hat{y}_i)^2$
Gradient	$\text{grad } J = [dJ/db_0, dJ/db_1]^T$
Gradient b_0	$dJ/db_0 = (2/n) * \sum_i (y_i - \hat{y}_i)$
Gradient b_1	$dJ/db_1 = (2/n) * \sum_i x_i (y_i - \hat{y}_i)$
Gradient Descent	$\theta_{t+1} = \theta_t - \alpha * \text{grad } J$
Finite difference	$dJ/d\theta \approx [J(\theta + \epsilon) - J(\theta - \epsilon)] / (2\epsilon)$
Hessian	$H = \nabla^2 J$
Stability (quadratic)	$0 < \alpha < 2/\lambda_{\max}(H)$
Standardization	$x_{\text{scaled}} = (x - \mu) / \sigma$
Condition number	$\kappa(H) = \lambda_{\max} / \lambda_{\min}$
Momentum	$v_{t+1} = \beta * v_t - \alpha * g_t; \theta_{t+1} = \theta_t + v_{t+1}$
Nesterov	$\theta_{\text{lookahead}} = \theta_t + \beta * v_t; g = \text{grad}(\theta_{\text{lookahead}})$
AdaGrad	$G_t = G_t + g_t^2; \theta_t \leftarrow \theta_t - \alpha * g_t / (\sqrt{G_t} + \epsilon)$
RMSProp	$S_t = \beta * S_t + (1 - \beta) * g_t^2; \theta_t \leftarrow \theta_t - \alpha * g_t / (\sqrt{S_t} + \epsilon)$
Adam first moment	$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * g_t$
Adam second moment	$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * g_t^2$
Adam bias correction	$\hat{m} = m / (1 - \beta_1^t), \hat{v} = v / (1 - \beta_2^t)$
Adam update	$\theta_t \leftarrow \theta_t - \alpha * \hat{m} / (\sqrt{\hat{v}} + \epsilon)$

26. What Was Completed / Stopping Point

The project now covers the complete optimization arc intentionally chosen for this Linear Regression module. The implementation stops at Adam, as planned.

Completed	Status
Linear Regression	Complete
MSE and residual analysis	Complete
Analytical gradients	Complete
Gradient Descent	Complete
Loss curves and animation	Complete
Loss surface / 2D contours / 3D	Complete
Gradient vector and gradient checking	Complete
Learning rate experiments	Complete
Hessian + eigenvalue stability	Complete
Feature scaling + condition number	Complete
Parameter trajectories	Complete
Momentum + beta study	Complete
Nesterov	Complete
AdaGrad	Complete
RMSProp	Complete
Adam	Complete

The next machine-learning algorithm can start as a separate module. This README/PDF does not extend the optimization section beyond Adam.

Final Takeaway

Loss -> Gradient -> Step size -> Curvature -> Conditioning -> Momentum -> Look-ahead -> Adaptive moments

The central lesson is that an optimizer is not just a loop over parameters. Its behavior is determined by the objective geometry, gradient information, step size, parameter scaling, and the optimizer state it chooses to retain. The Python files in this project make each of those ideas executable and inspectable.

Appendix A - Quick-start command block

```
cd C:\Users\Lenovo\machine-learning-algorithms\01_linear_regression
python -m pip install numpy matplotlib
python linear_regression.py
python loss_surface.py
python loss_surface_3d.py
python gradient_vector.py
python gradient_check.py
python learning_rate_experiment.py
python hessian_analysis.py
python stability_boundary.py
python feature_scaling.py
python parameter_trajectory.py
python momentum_gradient_descent.py
python momentum_beta_experiment.py
python nesterov_momentum.py
python adagrad.py
python rmsprop.py
python adam.py
```

Appendix B - Additional Plot Gallery

Additional plots reproduced from the user-provided plots.pdf (13 pages).

They are appended to the comprehensive Linear Regression & Optimization PDF

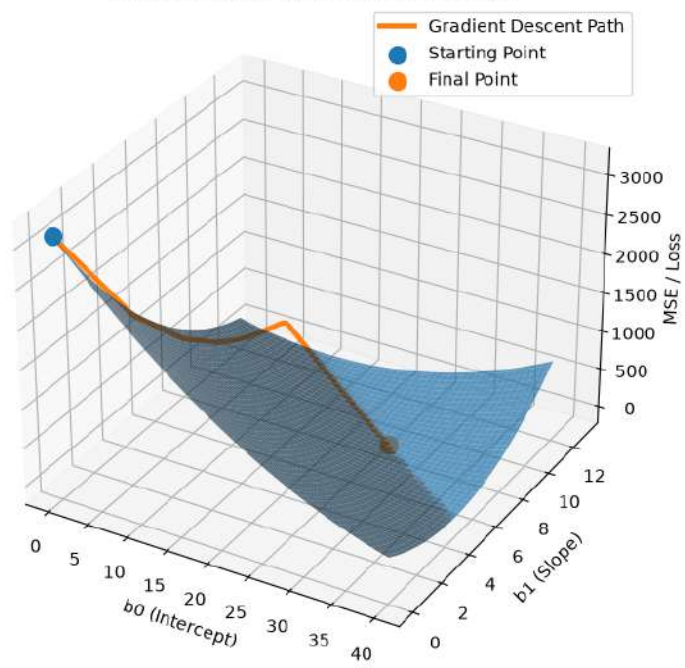
so that the document contains the complete visualization set, including

AdaGrad, RMSProp, Adam, gradient-vector, stability, momentum, and Nesterov plots.

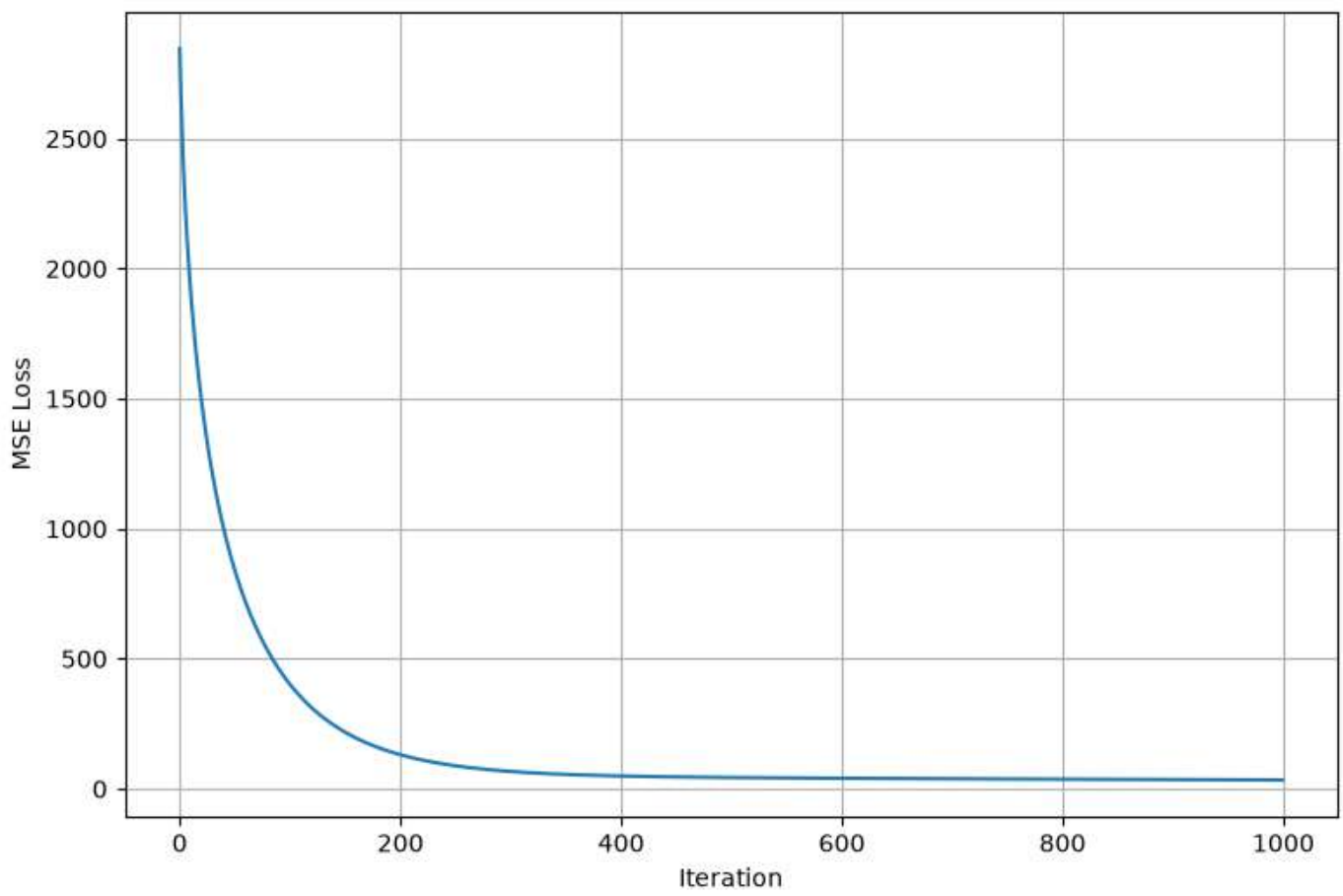
Appendix	Plot	Source
B-1	3D Loss Surface and Gradient Descent	plots.pdf page 1
B-2	AdaGrad - Loss vs Iteration	plots.pdf page 2
B-3	Adam - Loss vs Iteration	plots.pdf page 3
B-4	Feature Scaling - Loss vs Iteration	plots.pdf page 4
B-5	Gradient and Negative Gradient	plots.pdf page 5
B-6	Gradient Descent - First 30 Iterations	plots.pdf page 6
B-7	Gradient Descent Learning (snapshot)	plots.pdf page 7
B-8	Gradient Descent Near Stability Boundary	plots.pdf page 8
B-9	Gradient Descent Parameter Trajectory	plots.pdf page 9
B-10	Gradient Descent vs Momentum - Loss	plots.pdf page 10
B-11	Momentum Gradient Descent - Effect of beta	plots.pdf page 11
B-12	RMSProp - Loss vs Iteration	plots.pdf page 12
B-13	Vanilla GD vs Momentum vs Nesterov	plots.pdf page 13

Note: Appendix B preserves the plot pages as provided in plots.pdf; the original 19-page report remains unchanged.

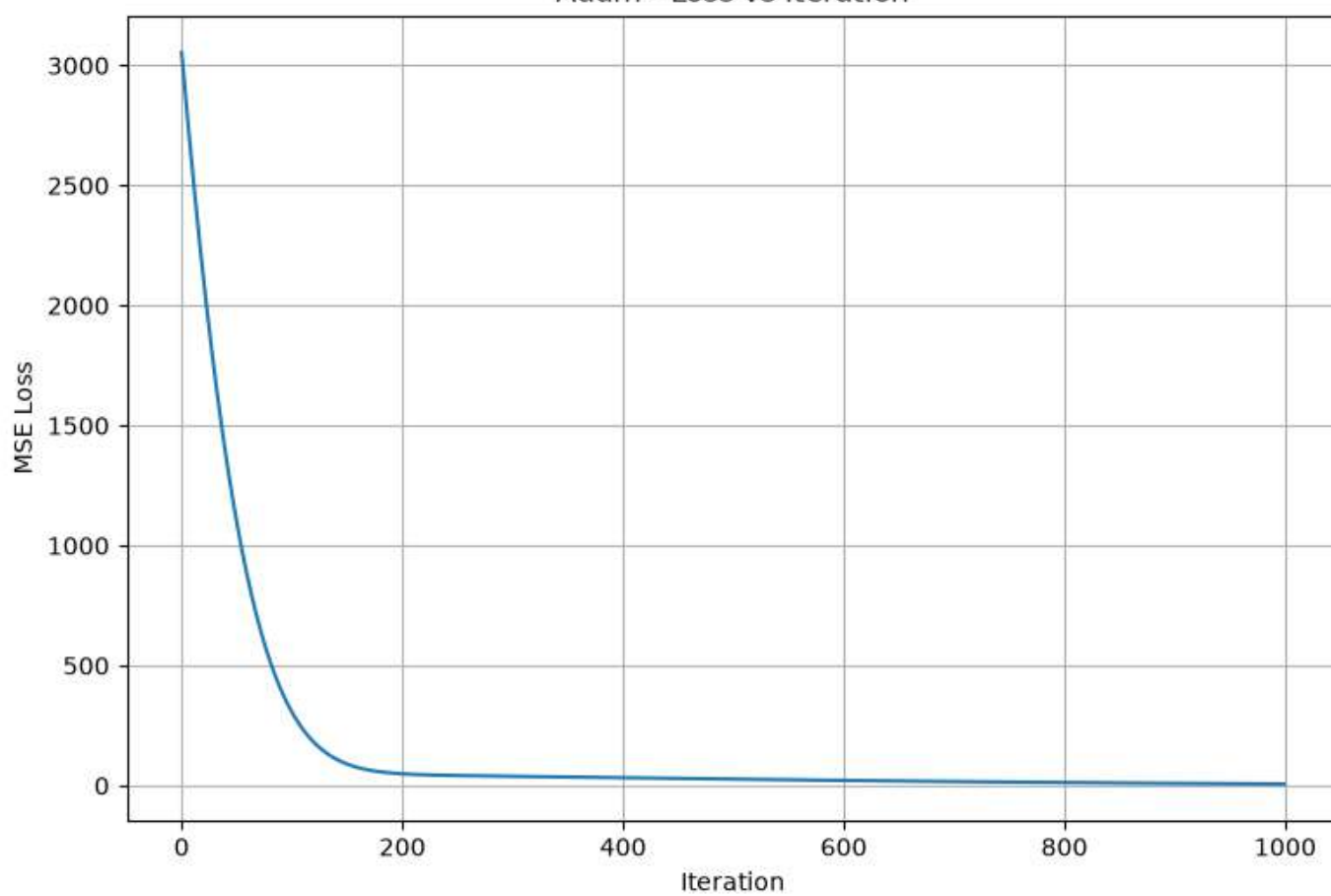
3D Loss Surface and Gradient Descent



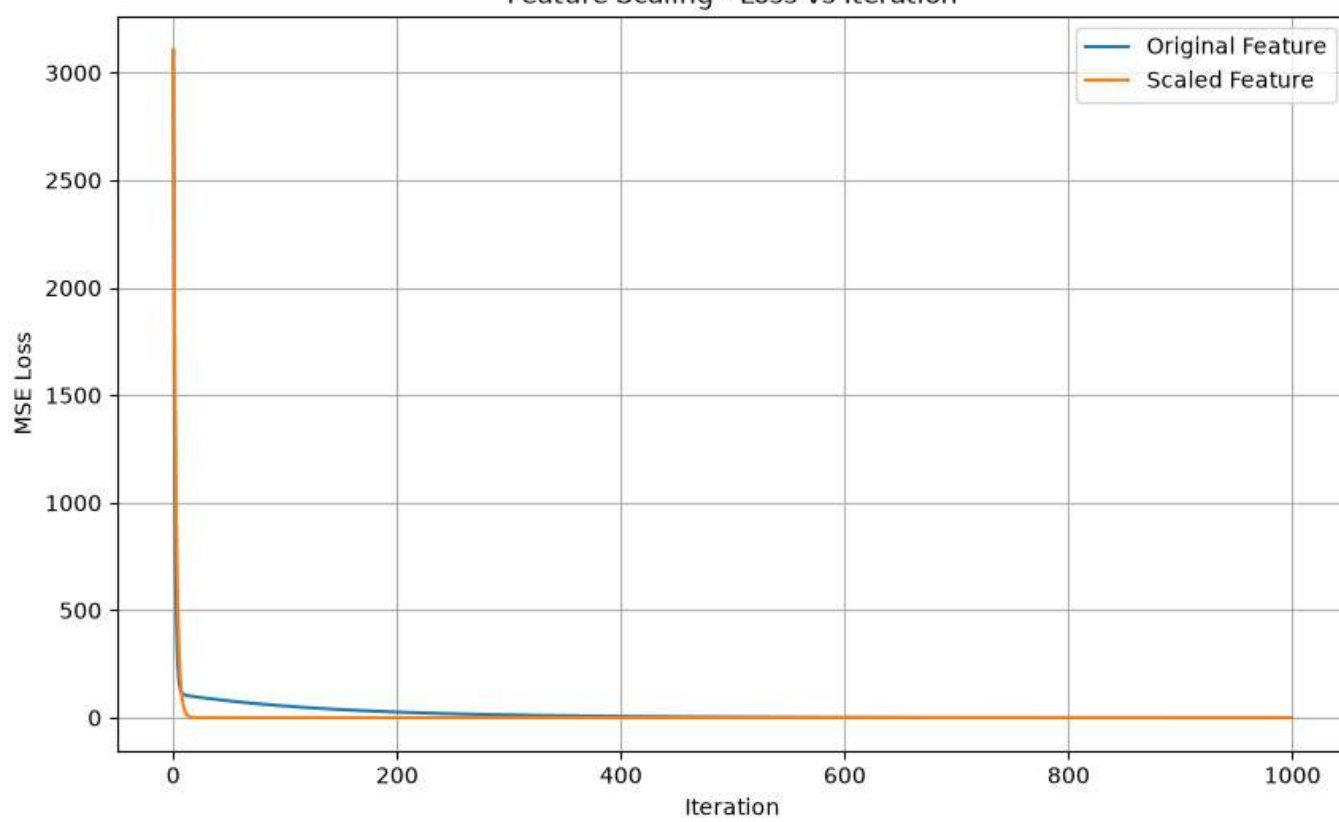
AdaGrad - Loss vs Iteration



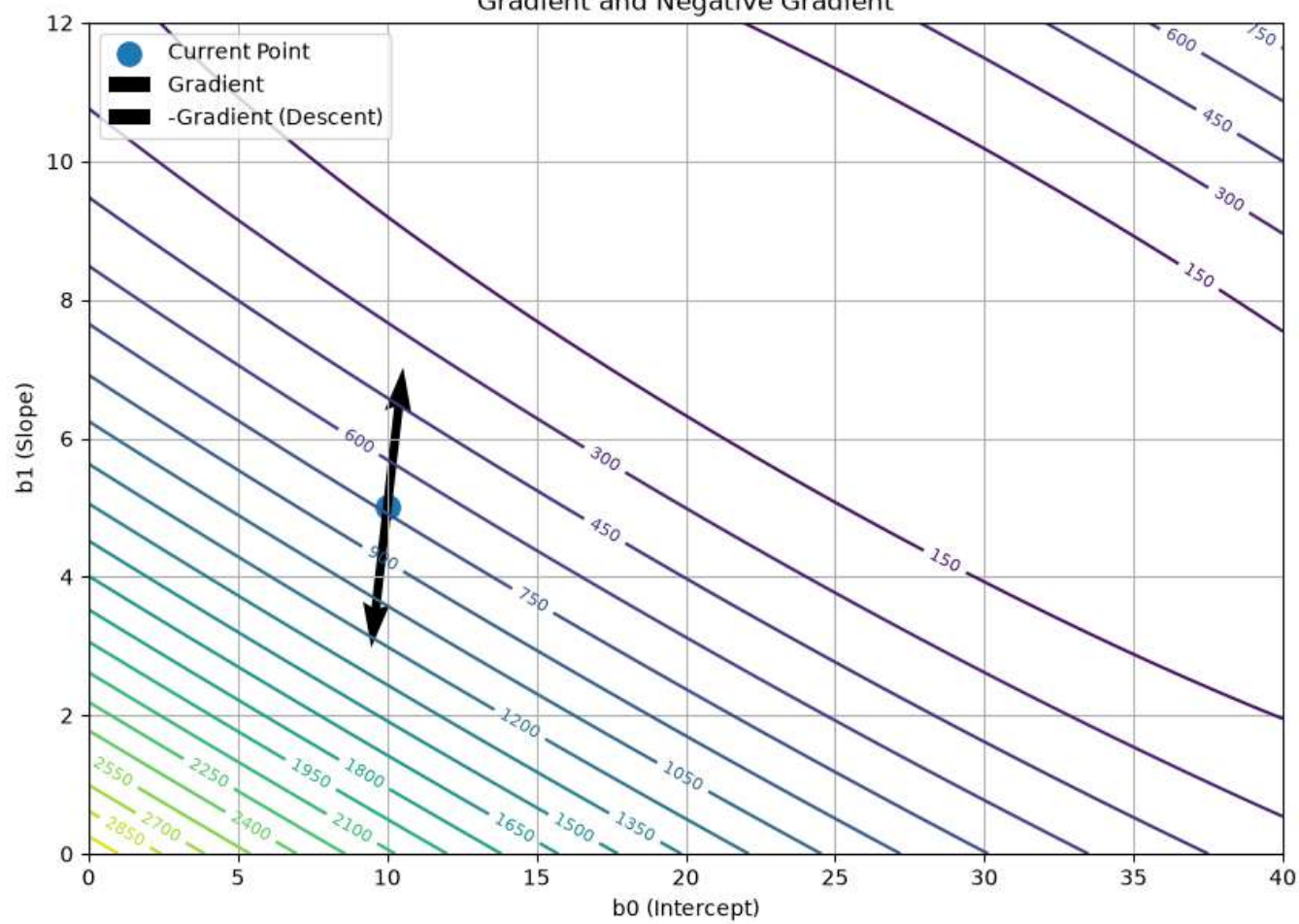
Adam - Loss vs Iteration

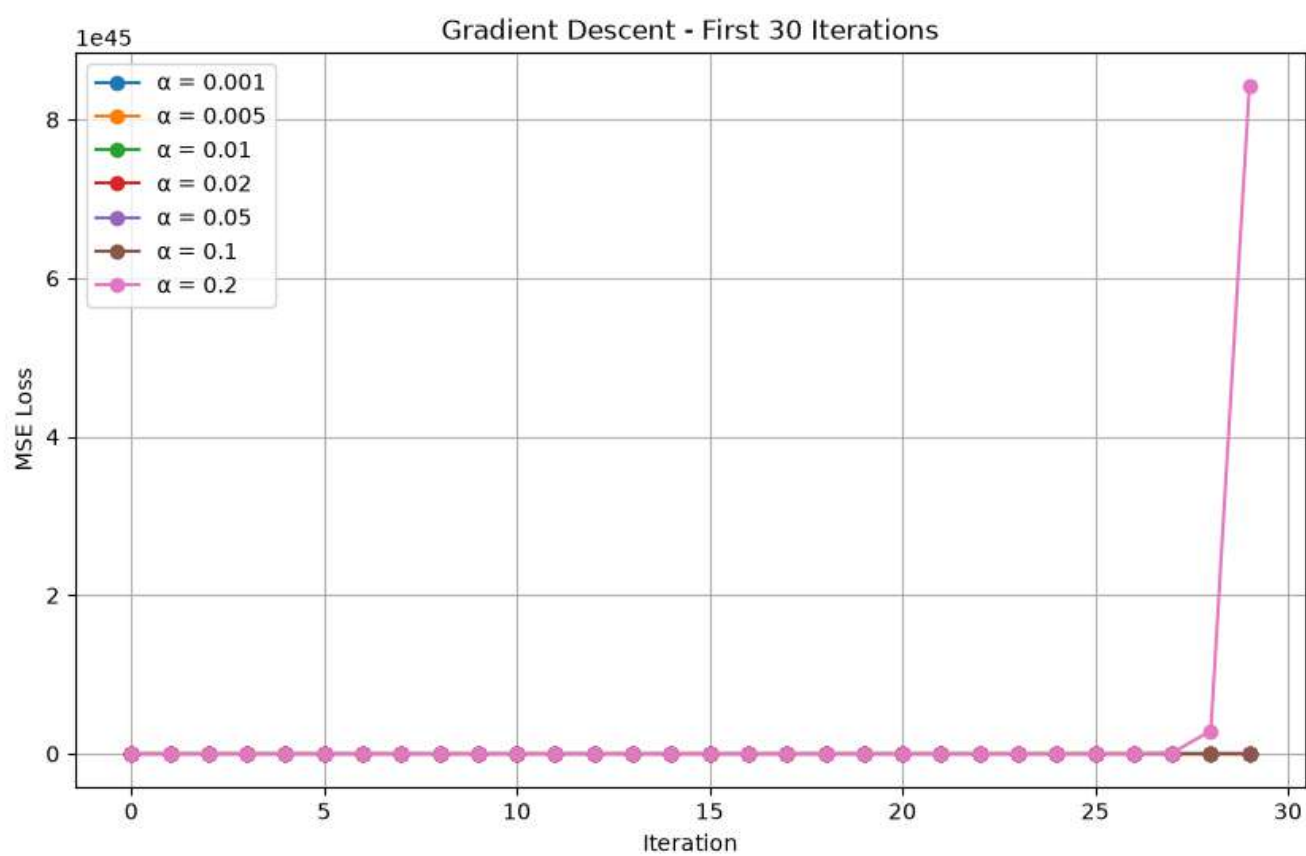


Feature Scaling - Loss vs Iteration

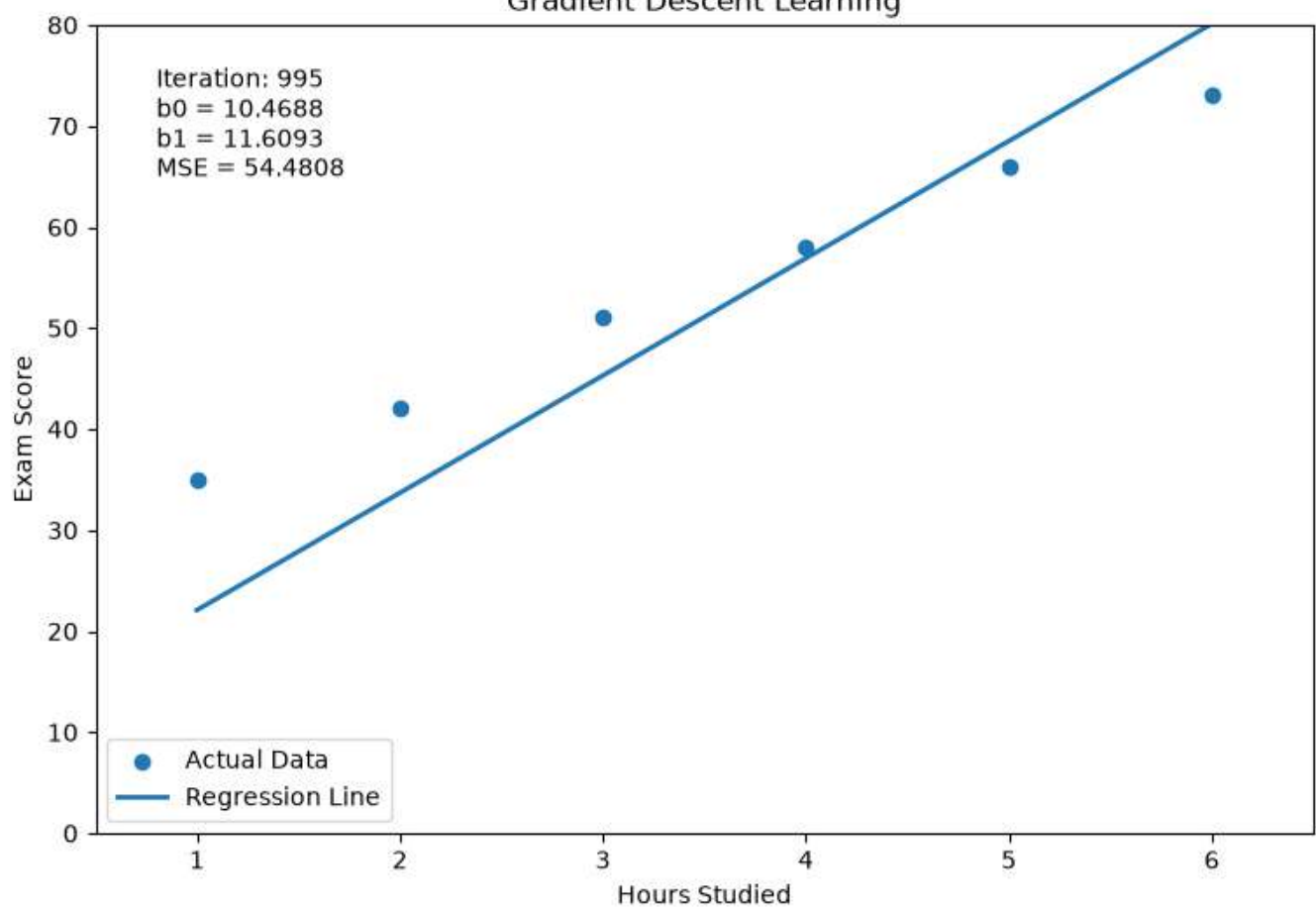


Gradient and Negative Gradient

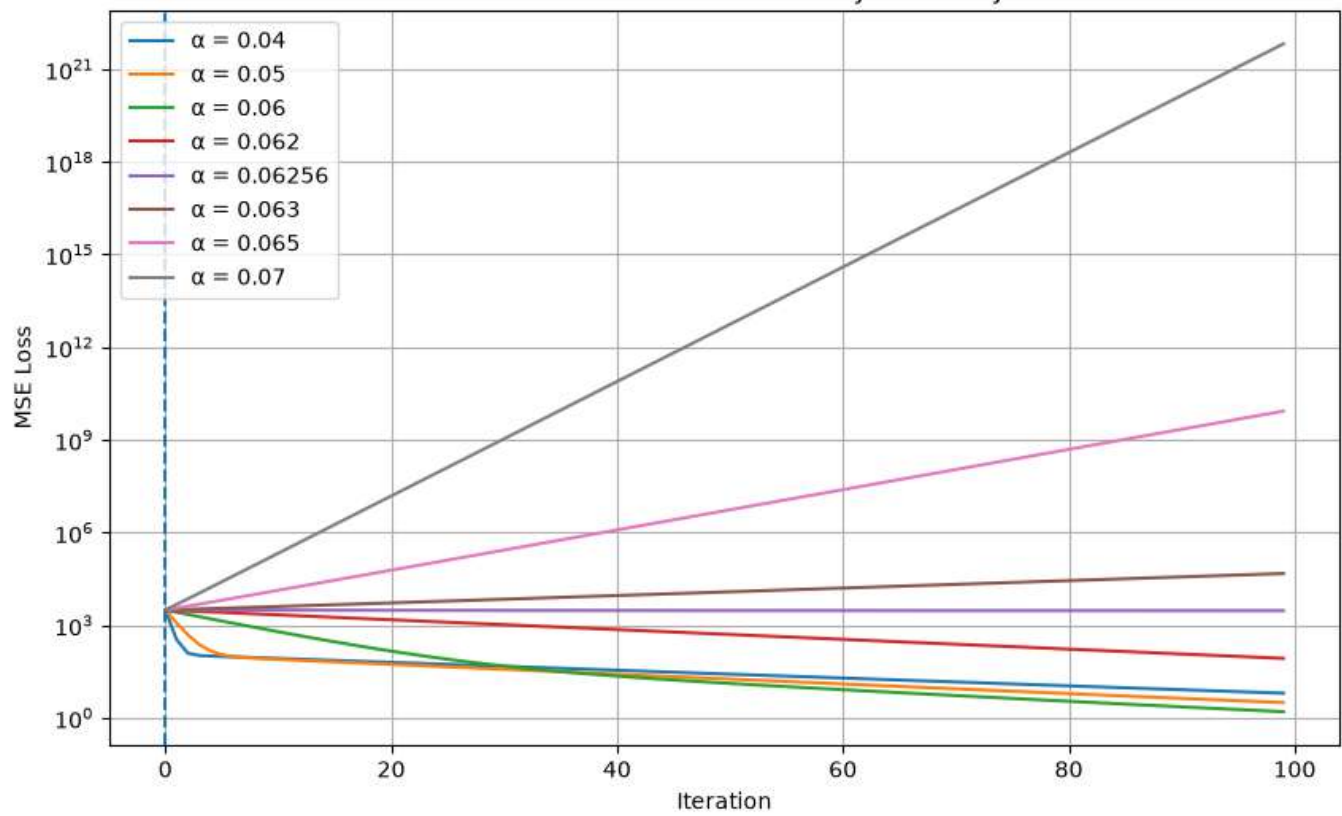




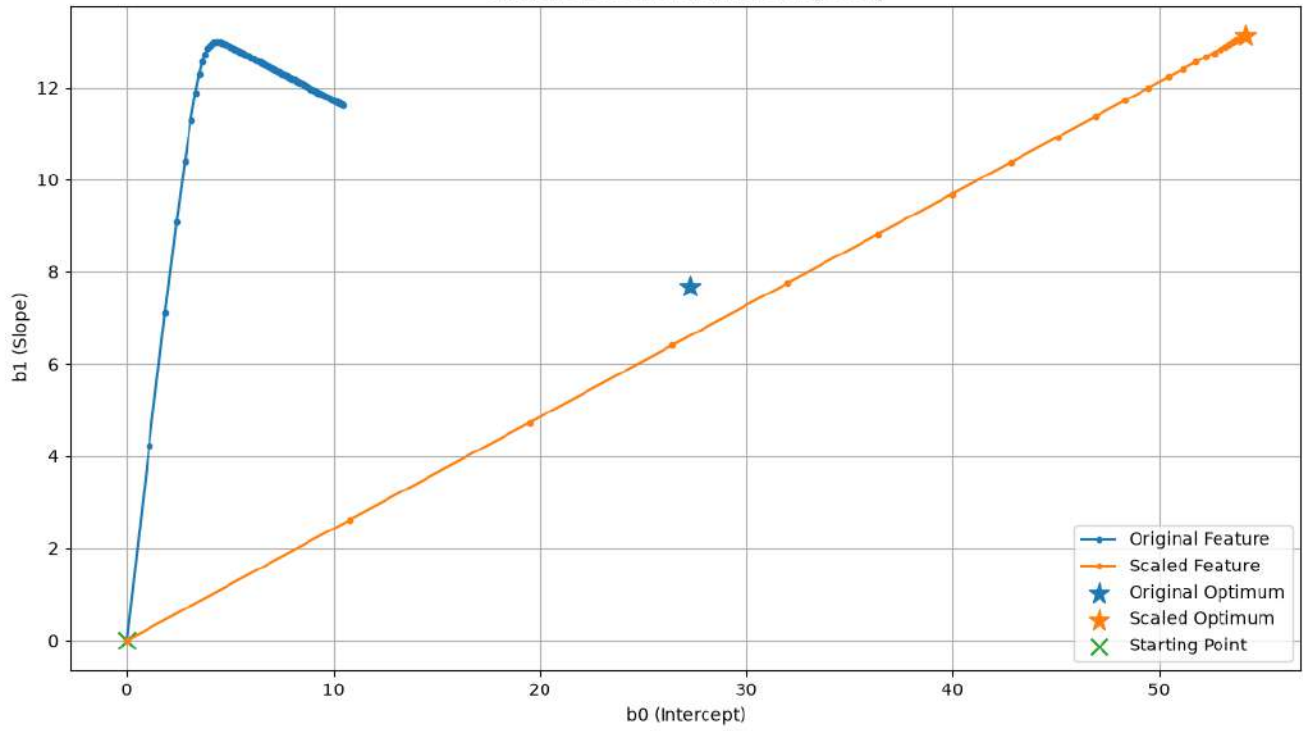
Gradient Descent Learning



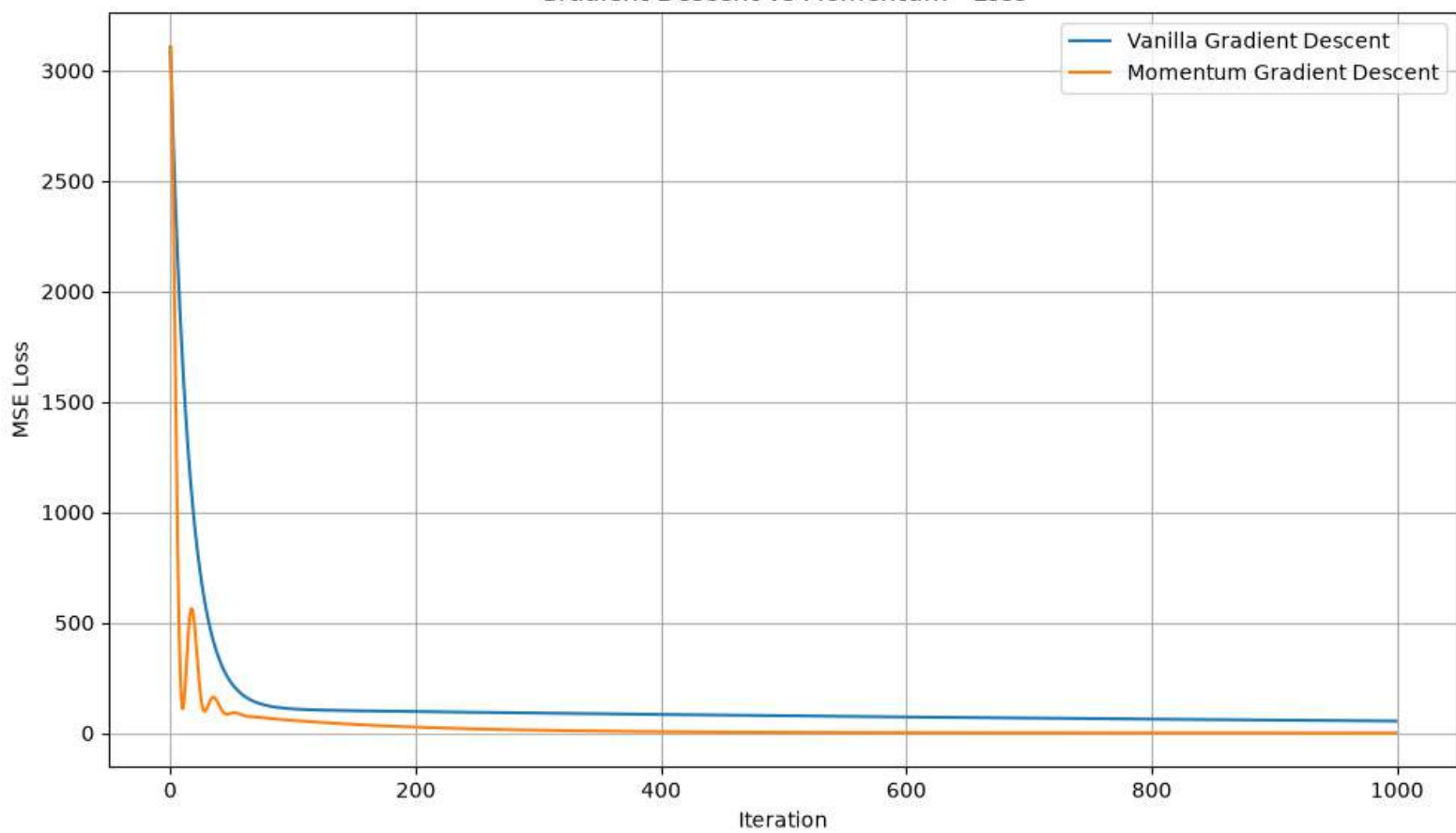
Gradient Descent Near Stability Boundary



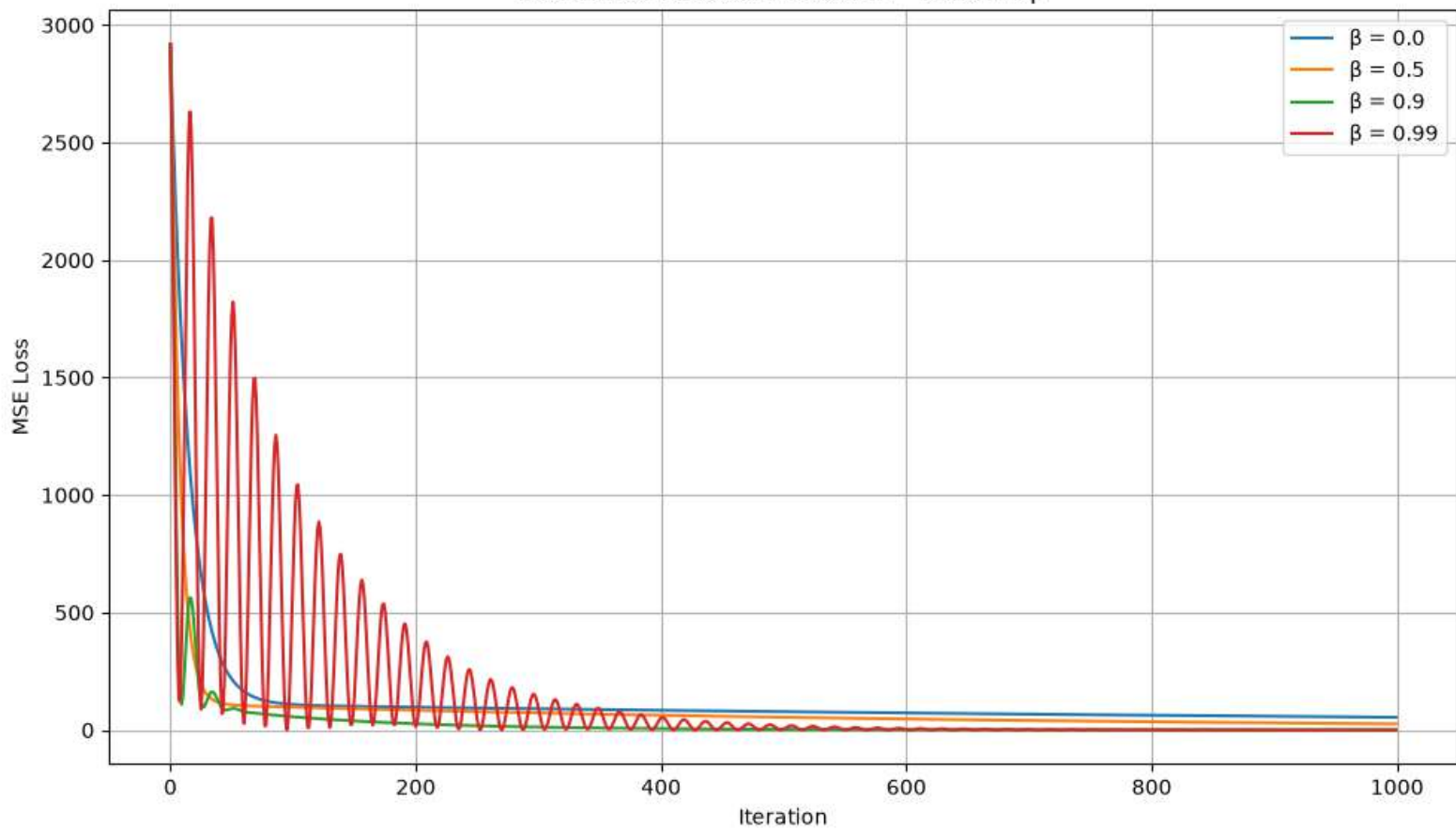
Gradient Descent Parameter Trajectory



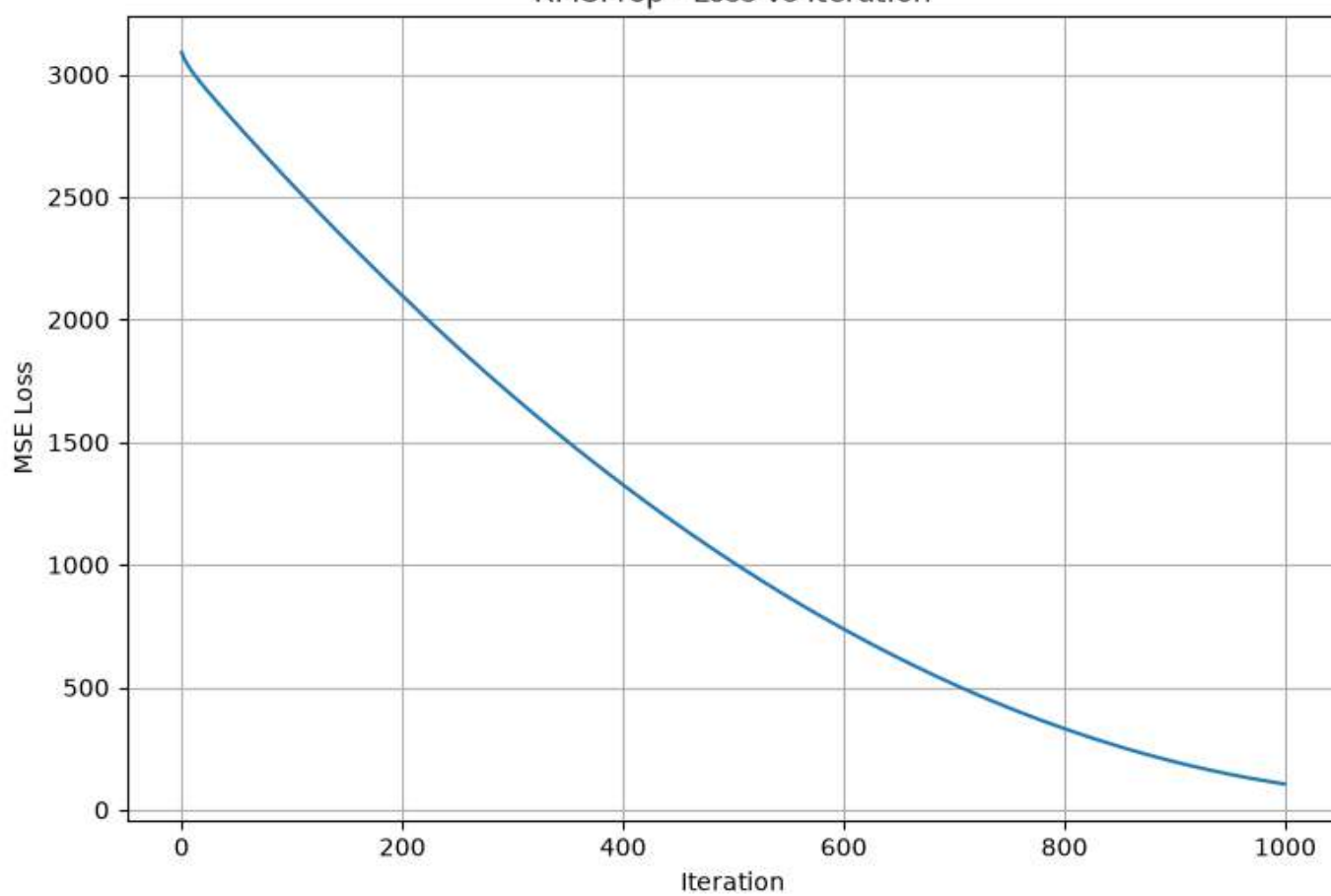
Gradient Descent vs Momentum - Loss



Momentum Gradient Descent - Effect of β



RMSProp - Loss vs Iteration



Vanilla GD vs Momentum vs Nesterov

